设计理念笔记

使用方式与目标

这份笔记的目标很简单：让你**写得稳、改得动、能讲清楚**。面试里最吃香的不是背定义，而是能把“为什么这样写”讲成一个清晰的故事：我面对的是怎样的场景，传统写法会带来什么问题，我用什么理念把问题收住，代价是什么，未来怎么扩展。

训练路线也按工程真实成长来：先把功能**跑起来**，再把结构变得**可维护**，最后在关键点上**留扩展点**，让后续需求来得再凶也不至于把代码推倒重来。

你可以把它当成“面试表达脚手架”。每个主题我都会尽量用同一套叙述节奏：先抛一个真实问题，再展示常见写法为什么会出事，然后给出理念与落地方式，最后用一段能讲清楚的 Python 例子收尾。

面向对象与建模

很多人第一次写“面向对象”，脑子里想的是“有类、有对象、有继承”。但工程里真正有用的面向对象，是把业务世界里**稳定的概念**和**变化的规则**，装进一组不容易被误用的模型里。换句话说，OOP 的价值不是“用类”，而是“用模型约束错误”。

为了把这个说清楚，我们从一个最常见的场景开始：下单。

你可以把“订单”当成一个普通的数据结构，然后在各处到处改字段；也可以把它当成一个带规则的对象，让所有状态变化都必须经过它的行为方法。两种写法都能跑，但第二种更容易让你在面试里解释清楚：**规则在哪里、为什么放那里、如何防止被绕开。**

领域建模的基本单位

第一次建模订单系统时，最“顺手”的写法通常是：建一堆类，里面一堆字段，谁需要就改字段。比如把 `order.status = "PAID"` 写在支付回调里，把 `order.total = sum(items)` 写在下单接口里。它的问题不在“能不能跑”，而在“跑久了会变成什么样”：当规则变多时，你会慢慢失去对一致性的控制，因为“谁在什么时候改了什么”散落在各个调用方里。

建模的第一步，是把领域里的概念按“它到底靠什么存在”分清楚。你不需要上来就背 DDD 大全，只要抓住几个直觉单位，面试里就能讲得很稳。

下面这个表把常用单位放到“它解决什么问题”的角度看，会更像工程语言，而不是概念背诵：

单位	你在解决的核心问题	典型例子	常见误区
实体（Entity）	“我是谁”——需要身份与生命周期	Order、User、Account	把所有东西都做成实体，导致到处是 ID、比较困难

单位	你在解决的核心问题	典型例子	常见误区
值对象（Value Object）	“它是什么值”——值相等即相等，适合不可变	Money、Address、TimeRange	把值对象做成可变，结果并发/推理都变复杂
领域服务（Domain Service）	“规则属于领域但不属于某个对象的天然职责”	Pricing、Eligibility	把领域服务写成万能工具类，最后变成上帝对象
工厂（Factory）	“创建过程复杂或要强制不变量”	create order、create account	只是为了“好看”而加一层，反而让创建更绕
仓储（Repository）	“把持久化细节隔离出领域层”	OrderRepository	在仓储里写业务规则，把边界打穿

在实践里，“实体还是值对象”最容易踩的坑，是把两者混用到让人分不清边界。一个很好用的自检问题是：**我在日志里、在数据库里、在接口里，需要稳定地指向它吗？**如果答案是“需要”，那它通常更像实体；如果答案是“我只在乎它的内容”，那它通常更像值对象。

另一个自检问题更贴地气：**我希望它是不可变的吗？**金额、地址、时间段这种东西，一旦在系统里被多个地方引用，做成不可变会让你少掉大量“谁改了它”的排查成本。你不需要把不可变当成教条，只要记住它的收益来自一个直觉：共享可变状态会让推理变难。

为了让“不变量应该放在哪里”更直观，可以用一句很口语的判断：**规则跟着数据走。**如果一条规则必须依赖订单的状态/明细/地址才能判断，那它更适合待在订单对象里；如果它必须同时看很多对象（订单 + 用户等级 + 活动配置），那它更适合落在领域服务里。把规则放到“离它需要的数据最近的地方”，通常就是最少重复、最少漏校验的地方。

实体和值对象的区别经常会被问成一句话：“什么时候用 ID，什么时候直接值比较？”你可以用一种非常工程化的表达来回答：当我需要追踪生命周期、需要在系统里长期引用它、需要区分“同值但不同个体”时，我用实体；当我只关心值的等价性、它不需要独立存在、复制成本低并且适合不可变时，我用值对象。

这些单位真正落地的关键点是**不变量（Invariant）**。

不变量是“无论系统怎么跑，这条规则都必须成立”。例如：订单总价不能为负；已支付订单不能再修改商品明细；地址一旦进入发货流程就不能随意改。最坏的做法是把不变量散在调用方里，因为任何一个漏掉校验的地方都会成为 bug 的入口。更稳的做法，是把不变量放进对象内部，通过行为方法统一保证。

把“不变量放对象里”听起来像一句口号，但它其实很具体：你要尽量减少“外面能直接改里面状态”的路径，让状态变化通过少数几个入口发生。下面这段 Python 就是一个最小可讲清的例子：订单是实体，金额/地址是值对象，定价是领域服务，创建用工厂，外部依赖通过协议抽象隔离。

```python
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
from __future__ import annotations
from dataclasses import dataclass
from decimal import Decimal
from enum import Enum
from typing import Protocol, Dict
import uuid


# ——— Value Object: Money ———
```

```python
@dataclass(frozen=True)
class Money:
    amount: Decimal
    currency: str = "CNY"

    def __post_init__(self) -> None:
        if self.amount < 0:
            raise ValueError("金额不能为负数")

    def __add__(self, other: "Money") -> "Money":
        if self.currency != other.currency:
            raise ValueError("币种不一致，不能相加")
        return Money(self.amount + other.amount, self.currency)


# ─── Value Object: Address ───
@dataclass(frozen=True)
class Address:
    city: str
    detail: str

    def __post_init__(self) -> None:
        if not self.city or not self.detail:
            raise ValueError("地址信息不完整")


class OrderStatus(Enum):
    DRAFT = "draft"
    SUBMITTED = "submitted"
    PAID = "paid"
    SHIPPED = "shipped"
    CANCELED = "canceled"


# ─── Entity: Order ───
class Order:
    def __init__(self, order_id: str, user_id: str, shipping_address: Address) -> None:
        self._id = order_id
        self._user_id = user_id
        self._shipping_address = shipping_address

        self._items: Dict[str, int] = {}  # sku -> qty
        self._status = OrderStatus.DRAFT
        self._total = Money(Decimal("0.00"))

    @property
    def id(self) -> str:
        return self._id

    @property
    def status(self) -> OrderStatus:
        return self._status

```

```python
    @property
    def total(self) -> Money:
        return self._total

    def add_item(self, sku: str, qty: int) -> None:
        # Invariant: 只有草稿/已提交状态可以改明细（示例规则）
        if self._status not in (OrderStatus.DRAFT, OrderStatus.SUBMITTED):
            raise ValueError(f"当前状态为 {self._status.value}，不允许修改商品明细")
        if qty ≤ 0:
            raise ValueError("数量必须为正数")
        self._items[sku] = self._items.get(sku, 0) + qty

    def change_address(self, new_address: Address) -> None:
        # Invariant: 发货后不允许修改地址
        if self._status == OrderStatus.SHIPPED:
            raise ValueError("已发货订单不允许修改地址")
        self._shipping_address = new_address

    def submit(self, pricing: "PricingService") -> None:
        # Invariant: 只有草稿状态允许提交（避免重复提交/重复计价）
        if self._status ≠ OrderStatus.DRAFT:
            raise ValueError(f"当前状态为 {self._status.value}，不允许再次提交")

        # Invariant: 空订单不允许提交
        if not self._items:
            raise ValueError("订单没有商品，不能提交")
        self._total = pricing.calculate_total(self._items)
        self._status = OrderStatus.SUBMITTED

    def mark_paid(self) -> None:
        # Invariant: 只能从已提交到已支付
        if self._status ≠ OrderStatus.SUBMITTED:
            raise ValueError("订单未提交，不能支付")
        self._status = OrderStatus.PAID

    def mark_shipped(self) -> None:
        if self._status ≠ OrderStatus.PAID:
            raise ValueError("未支付订单不能发货")
        self._status = OrderStatus.SHIPPED


# ─── Domain Service: pricing rules ───
class PricingService:
    def __init__(self, catalog: "ProductCatalog", discount: "DiscountPolicy") -> None:
        self._catalog = catalog
        self._discount = discount

    def calculate_total(self, items: Dict[str, int]) -> Money:
        total = Money(Decimal("0.00"))
        for sku, qty in items.items():
            price = self._catalog.price_of(sku)
            total = total + Money(price.amount * qty, price.currency)
        return self._discount.apply(total)
```

```python


# ─── Boundary abstractions (external dependencies) ───
class ProductCatalog(Protocol):
    def price_of(self, sku: str) -> Money:
        ...


class DiscountPolicy(Protocol):
    def apply(self, total: Money) -> Money:
        ...


# ─── Factory: create Order in a valid way ───
class OrderFactory:
    def create(self, user_id: str, address: Address) -> Order:
        order_id = str(uuid.uuid4())
        return Order(order_id=order_id, user_id=user_id, shipping_address=address)
```

很多“面向对象讲不清”的面试，卡点其实就在这里：你说你用了实体/值对象，面试官会追问“有什么用”；你说你封装了状态，面试官会追问“封装到底是什么”。所以我们紧接着把封装讲透。

封装的真正含义

很多人把封装理解成“private 字段 + getter/setter”。这就是为什么面试官爱追问：这种理解太表面。封装真正要解决的是**把状态变化的入口收口**，让对象对外暴露的是“能做什么”，而不是“有什么字段”。

如果订单对外暴露 `set_status()` 、 `set_total()` 、 `set_items()` ，调用者就会在任何地方随手改状态。你很难保证所有调用者都记得检查“已支付不能改明细”这样的规则。最后代码会变成一种很典型的坏味道：每个接口里都有一坨 if 校验，每一坨还不完全一样，甚至某些路径根本没校验。

所以封装的关键动作不是“不给你改”，而是“我只让你通过正确的行为去改”。比如上面 `mark_paid()` 强制要求状态必须是 `SUBMITTED` ，这比在支付接口里写一遍校验要更稳，因为任何路径想把订单改成已支付，都必须走这个入口。

封装带来的第二个好处，是一致性校验会变得“有家可归”。范围校验（金额不能为负、数量必须为正）、互斥校验（某些状态不能同时成立）、状态机约束（从 A 才能到 B）这些都应该尽量靠近拥有状态的对象。靠近的意思是：规则写在能看见状态的地方，而不是散在调用方的流程里。

这里可以用一句“面试可复用”的总结句式：封装不是隐藏数据，而是隐藏**状态变化的方式**；你不怕别人看见你的字段值，你怕的是别人绕过你的规则。

为了让“入口收口”更可见，我们补一段“传统写法 vs 封装写法”的对比。注意它并不追求完整，而是追求让问题显形。

```python
# 传统写法：对象像字典，谁都能改，规则散落
order.status = "PAID"            # 某个支付回调里
order.items["sku_1"] += 1        # 某个运营改单接口里
# 结果：你只能靠“到处 if”祈祷不会漏

# 封装写法：对象像一个小系统，状态变化必须走入口
order.mark_paid()               # 入口自己保证：必须从 SUBMITTED 到 PAID
order.add_item("sku_1", 1)      # 入口自己保证：哪些状态允许改明细
```

当你能把这一段解释清楚，封装在面试里基本就“稳了”。接下来最常见的追问是：那你怎么复用？是不是都要靠继承？这就进入下一节。

继承的边界与组合优先

继承的诱惑来自“我只要继承一下就能复用代码”。但工程里最贵的不是写代码，是改代码。继承最容易把系统变成“改一个地方，满世界都受影响”，尤其是当你用继承表达“业务分支”时。

面试里常用的判断是 IS-A：语义上它真的是一种“是”的关系吗？更关键的是替换原则：父类能出现的地方，子类都能安全替换吗？如果答案勉强，继承就会变成未来的坑。

一个典型失败例子是“用继承做业务组合”。比如你想做不同折扣类型，于是写 `VipOrder(Order)`、`CouponOrder(Order)`、`FlashSaleOrder(Order)`。一开始很爽，后来规则交叉：既能用券也能是 VIP，还要叠加活动上限，继承树立刻崩掉。你会发现你在用类层级表达组合关系，而组合关系天生更适合用“组合”。

组合优先的直觉是：把变化点抽出来，让对象把它“带在身上”而不是“长在骨头里”。在上面的例子里，折扣就是一个变化点，所以我们没有让订单继承出各种子类，而是引入了 `DiscountPolicy`。这就是策略模式的最小形态：订单提交时，定价服务通过一个可替换的策略来计算最终价格。
区分一句话：**策略**通常是“同一目标，多种算法可互换”；**状态**通常是“同一对象，随内部状态变化而切换行为”，切换由对象自己驱动。

下面补上两个折扣策略，你就能很直观地感受到它的扩展方式：加一个新折扣，不需要改订单类，也不需要改定价服务的核心流程，只要新增策略实现。面试里你可以顺势说清楚代价：多了一个抽象层，需要管理策略的选择；但收益是替换成本低、代码更不容易被连坐。

```python
from decimal import Decimal

class NoDiscount:
    def apply(self, total: Money) -> Money:
        return total


class TenPercentOff:
    def apply(self, total: Money) -> Money:
        return Money(total.amount * Decimal("0.9"), total.currency)
```

到这里，第一章的核心就形成了一个闭环：用实体和值对象把概念稳定下来，用封装把不变量收住，用组合把变化点隔离。接下来第二章会把视角从“一个对象内部”拉到“多个模块之间”，也就是分层、边界和依赖管理。

模块化与依赖管理

面试里聊到“架构/系统设计”，经常会出现一种尴尬：你能把业务流程说清楚，但代码怎么组织、依赖怎么控制，说不出一个稳定的方法。模块化与依赖管理的目的，本质上就是一句话：**把变化关在边界里**，让变化不至于把整个系统拖着走。

这里依然从一个非常现实的问题开始：你写了订单提交逻辑，里面要查商品价格、要写数据库、要发消息。最直觉的写法是：在一个函数里把这些都干了。它的短期收益很明显：快。它的长期代价也很明显：难测、难改、难替换。

当你把外部依赖（数据库、HTTP、消息队列、第三方支付）直接写死在核心逻辑里，领域模型就会被“外界的噪声”淹没。你以后想改数据库、想做本地测试、想加缓存、想把消息从 Kafka 换成别的，都会牵一发动全身。

第二章要解决的就是这类问题：我怎么分层？怎么划边界？接口到底什么时候值得引入？DI 为什么能提升可测试性？我们会用尽量少的术语把这些讲成工程直觉。

分层与边界

分层不是为了画漂亮的图，而是为了隔离变化。你可以把系统想成两类东西：一类是“业务规则”，一类是“与世界打交道的手段”。业务规则希望稳定，打交道的手段经常变化。

一个常见、好解释的分层方式是：UI（或 API）负责接收请求与返回响应，应用层负责编排用例（调用哪些领域能力），领域层负责规则与模型，基础设施层负责数据库、消息、第三方服务等“怎么做”的细节。你也可以简化成三层，只要你能讲清楚边界在哪里。

分层最重要的不是层名，而是依赖方向：**高层不依赖低层的具体实现**，高层只依赖抽象。这样，低层的实现可以替换，高层不用改。

为了不让它停留在口号，我们用一个最小例子把“边界”做出来：下单提交的用例（应用层）需要保存订单（仓储）并计算价格（定价服务）。仓储是外部依赖（基础设施），所以用协议（抽象）把它隔离。

```python
# 注：此代码块可能含占位符（ ... ），用于说明结构；如需运行请补齐实现
from typing import Protocol, Optional


class OrderRepository(Protocol):
    def get(self, order_id: str) -> Optional[Order]:
        ...

    def save(self, order: Order) -> None:
        ...


class SubmitOrderUseCase:
    # 这是应用层：编排用例，不放业务细节
    def __init__(self, repo: OrderRepository, pricing: PricingService) -> None:
        self._repo = repo
        self._pricing = pricing

    def execute(self, order_id: str) -> None:
        order = self._repo.get(order_id)
        if order is None:
            raise ValueError("订单不存在")

        # 业务规则仍在领域对象里（Order.submit），用例只负责协调
        order.submit(self._pricing)
        self._repo.save(order)
```

如果面试官把话题上升到“你这属于什么架构”，你可以用一个非常省力的对齐：你这里的分层和 Clean Architecture / 六边形（Hexagonal）/ 端口适配器（Ports & Adapters）是同一类思想。Domain 在中心，外部依赖（DB、MQ、第三方）在外圈；`OrderRepository`、`PaymentGateway` 这些协议就是“端口（port）”，数据库实现、SDK 适配就是“适配器（adapter）”。你不需要画复杂图，只要说清楚依赖方向：核心只依赖抽象，细节实现向抽象靠拢。

另外一个国内高频追问是“事务在哪开？”比较稳的回答是：应用层（UseCase）负责用例编排与事务边界，领域层负责规则与不变量，基础设施层提供事务机制（比如数据库会话/事务管理器）。也就是说，事务通常跟着“用例”走，而不是散落在领域对象里。

你可以注意到一个很关键的味道：`SubmitOrderUseCase` 并不知道数据库是什么，也不知道 ORM 是什么，它只知道“我有一个能取/存订单的东西”。这就是边界带来的可替换性。你做单测时可以用内存仓储，线上用数据库仓储，用例层不需要改。

这一节如果面试官追问“那为什么要分层”，你可以用三个非常朴素、但很有效的回答：

- 分层让变化有地方放：外部依赖变了，尽量只改基础设施层。
- 分层让测试更简单：核心逻辑不用启动数据库就能测。
- 分层让协作更清楚：不同人负责不同边界，互不踩脚。

接下来会自然进入依赖倒置：为什么高层依赖抽象，低层提供实现；以及接口到底什么时候该引入、什么时候不该滥用。

依赖倒置与接口

在第二章里，“依赖倒置”常常会被误解成“多写接口”。其实它关心的是一句很朴素的话：**变化多的东西不要支配变化少的东西**。数据库、第三方 SDK、网络调用这些东西变化快；你的业务用例和领域规则变化相对慢。你不希望慢的东西被快的东西绑架。

传统写法里，最典型的绑定方式就是“用例层直接 new 数据库客户端”。你写完会很快，但很快也会遇到两个现实麻烦。第一是测试：你想测一个用例，却不得不连数据库、连外部服务，测试变慢、变脆，失败时还不确定是逻辑错还是环境抖。第二是替换：你想把 MySQL 换成 PostgreSQL、把某个 SDK 换个版本、甚至只是加一层缓存，都会把改动带到核心逻辑里。

依赖倒置的思路是把依赖方向反过来：高层（用例/领域）只依赖抽象，低层（数据库/SDK）去实现这些抽象。于是“边界”就出现了：边界之内是你想稳定的业务表达，边界之外是可以替换的实现细节。这样你新增一个实现，是“加文件”，而不是“改核心”。

那接口到底什么时候值得引入？你可以用三个很实用的判断：

- 当你需要替换点：同一职责存在多个实现，或者未来大概率会出现多个实现（比如多种消息渠道、多种存储）。
- 当你需要隔离不稳定边界：第三方 SDK、网络、时间、随机数、文件系统这类东西，通常都适合用抽象隔离。
- 当你需要更好地测试：你希望核心逻辑在单测里用内存实现或假实现，而不是启动真实依赖。

什么时候不需要接口也要敢说清楚：如果它是内部稳定实现，并且你非常确定不会替换，也不会影响测试，那就直接用类。过度抽象的坏处是真实存在的：多一层跳转、多一套命名、多一堆“空接口”，新同事读起来会更慢。

至于“抽象类 vs 接口”，你可以用一句对比讲清：抽象类更像“带默认实现的半成品模板”，强调代码复用；接口更像“行为契约”，强调可替换与边界。Python 里没有强制接口，但 `Protocol` 让你可以用结构化类型表达契约，它的好处是：写起来轻，不会把系统锁死在继承树上。

仓储（Repository）在国内面试里也经常被追问一个很实操的问题：**复杂查询放哪？**一个比较稳的回答是：写模型（领域）侧的仓储应该表达意图，尽量别把领域对象拖成 ORM 查询容器；当查询变得非常复杂、且主要用于展示（列表页、报表、搜索）时，可以把它放到“查询服务/读模型”里，甚至做一个轻量的 CQRS：写侧保持领域模型干净，读侧用更贴展示需求的数据结构（可能是 join 结果、视图、缓存后的 DTO）。你不是在炫概念，而是在承认读写关注点不一样，避免一个模型两头都难受。

下面用一个很小的例子把依赖倒置落地：用例层只依赖 `OrderRepository` 协议；具体实现（比如数据库仓储）留到基础设施层。这让测试可以用 `InMemoryOrderRepo` ，线上可以用 `SqlOrderRepo` ，用例完全不需要改。

```
# 注：此代码块可能含占位符（ ... ），用于说明结构；如需运行请补齐实现
from typing import Optional, Protocol, Dict

class OrderRepository(Protocol):
    def get(self, order_id: str) -> Optional[Order]: ...
    def save(self, order: Order) -> None: ...

class InMemoryOrderRepo:
    def __init__(self) -> None:
        self._data: Dict[str, Order] = {}

    def get(self, order_id: str) -> Optional[Order]:
        return self._data.get(order_id)

    def save(self, order: Order) -> None:
        self._data[order.id] = order
```

当面试官追问“你这不就是多写了点代码吗”，你可以把收益说得很工程：核心逻辑变得可测试、可替换、可演进，而不是被外部依赖的细节拖着走。

依赖注入与生命周期

当你把依赖倒置做出来以后，下一步自然会遇到一个问题：既然用例类不再自己 new 依赖，那这些依赖到底从哪里来？这就是依赖注入（DI）要解决的事。你可以把 DI 想成一种“组装方式”：对象只声明自己需要什么，组装者把依赖塞给它。

传统做法是“在对象内部创建依赖”。它最大的问题不是丑，而是让依赖变成隐式的。你看到一个 `SubmitOrderUseCase()` ，你并不知道它内部到底 new 了什么；你想换个实现，也很难。更糟糕的是测试：你没法轻易把真实依赖换成假的依赖。

构造注入之所以被推荐，是因为它把依赖写在构造函数上——依赖变成显式的、可见的、可替换的。一个类如果需要五六个依赖，这并不是 DI 的错，而是在提醒你：这个类的职责可能太大了。DI 反而让坏味道更早暴露。

生命周期则是 DI 里最容易被问但又不需要背太细的点。你只要抓住直觉：有的对象应该每次请求都新建（避免共享可变状态），有的对象可以在一次请求内复用（比如缓存一次查询结果），有的对象可以全局复用（比如只读配置、线程安全的连接池）。在很多后端框架里常见的词是 Transient/Scoped/Singleton，你不需要死记定义，只要能解释“为什么这个对象适合这个生命周期”。

这里有一个非常高频的坑，面试官也爱问：**Singleton 依赖 Scoped**。直觉解释是：如果一个全局对象里握着“每次请求都不同”的东西，那它就会把不同请求的状态串起来，产生数据串扰或并发 bug。类似地，全局静态状态也会污染测试：上一个测试留下的状态会影响下一个测试。

下面这段代码用最朴素的方式展示 DI：我们在“组装点”创建依赖并注入用例。你可以把组装点理解成 main、框架的容器、或者应用启动时的 wiring。

```
# 注：此代码块可能含占位符（ ... ），用于说明结构；如需运行请补齐实现
from dataclasses import dataclass
from typing import Protocol, Dict

```

```python

class Catalog(Protocol):
    def price_of(self, sku: str) -> int: ...


class DiscountPolicy(Protocol):
    def apply(self, sku: str, subtotal: int) -> int: ...


@dataclass(frozen=True)
class FixedCatalog:
    prices: Dict[str, int]

    def price_of(self, sku: str) -> int:
        return self.prices[sku]


class NoDiscount:
    def apply(self, sku: str, subtotal: int) -> int:
        return subtotal


@dataclass
class PricingService:
    catalog: Catalog
    discount: DiscountPolicy

    def quote(self, sku: str, qty: int) -> int:
        subtotal = self.catalog.price_of(sku) * qty
        return self.discount.apply(sku, subtotal)


# 组合（这就是“注入”）：外部把依赖拼好再交给业务对象
service = PricingService(
    catalog=FixedCatalog({"APPLE": 3, "BANANA": 2}),
    discount=NoDiscount(),
)

print(service.quote("APPLE", 5))  # 15
```

面试表达上，你可以用一句话收尾：DI 不是为了“用框架”，而是为了让依赖显式、让测试容易、让替换可控。框架容器只是把这件事自动化了。

多环境与配置注入也是国内面试常问点。一个简单但很稳的原则是：配置在启动时读取，尽量做成不可变对象，然后通过 DI 注入到需要它的地方。这样测试可以注入测试配置，线上可以注入生产配置；同时你避免了“到处读环境变量/全局单例配置”的隐式依赖，排障和复现都会更容易。

设计模式

设计模式在面试里经常被问到，但真正加分的不是“我记得 23 个名字”，而是你能把它讲成一段自然的推理：我遇到一个真实问题，先用直觉写法解决，发现它会在扩展、复用、测试、协作上出问题，于是我把问题抽象成一个“变化点”或“边界”，再用某个模式把这个变化点固定住。

这一章我保留表格与示例代码，因为它们适合“落地”。但在每个表格之前，我都会先用 3～4 段自然段把模式“从哪里来”讲清楚：问题是什么、传统思路怎么做、为什么会出事、设计模式的思路怎么把问题收住。你在面试里几乎可以直接照这个节奏讲。

面试里最常见的问法其实不是“解释一下某某模式”，而是“你遇到过什么场景，用了什么办法，为什么这么选”。所以你在讲模式时，最好先占住“问题”这一侧：我当时面对的约束是什么（扩展频繁？外部依赖不稳定？对象组合爆炸？状态机复杂？），然后再说“我把变化点抽出来”，最后落到模式名字。这样面试官会感觉你是在做工程设计，而不是在背百科。

还有一个很好用的小技巧：当你不确定某个模式是不是“该用”，就问自己一句话——**我现在最痛的是重复、还是耦合、还是扩展成本？**模式本质上是在不同的痛点之间做取舍：有时你用一层抽象换来可替换性，有时你用组合换来扩展性，有时你用集中入口换来更稳定的边界。能把“换来什么、付出什么”讲清楚，模式就讲清楚了。

你在讲每个模式时，可以固定用一个“抽象句式”做收束，面试官会很舒服：**我把 X（变化/复杂性/不稳定依赖）从 Y（核心流程/领域规则）里抽出来，放到 Z（可替换点/边界/组合结构）**。这句话一出，模式名字反而变成“注释”，而不是你表达的主体。

另外，国内面试更吃“真实工程对应物”。所以在每个模式的引入里，我会尽量顺带点一下它在业务里的常见落点：比如责任链像 middleware/校验链，装饰器像缓存/重试/鉴权，观察者像领域事件/消息订阅，命令像任务队列/审计与重放。你不需要夸大经历，只要能把模式和工程场景对齐，可信度就会很高。
区分一句话：**装饰器**强调“在不改原类的前提下叠加能力”；**代理**强调“控制访问/延迟加载/隔离远程或昂贵资源”，增强只是手段。

创建型

单例模式（Singleton）

你可以想象一个小团队共用一台“打印机”。如果每个人都各买一台，成本高；但如果每个人都能随手改打印机设置、随手重置驱动，那所有人都会被影响。这里的关键不是“打印机只能有一台”，而是“打印机是共享资源，需要统一管理”。

传统写法在代码里很像：谁要用配置/缓存就自己 new 一个。短期看很自由，长期会出现两个典型问题：一是重复初始化（慢、浪费），二是多个实例状态不一致（我改了配置你没看到）。于是你开始把它做成“全局变量”，结果又遇到新的坑：全局可变状态让测试很难隔离，依赖变成隐式的，出问题时也不好定位。

单例模式的思路是：我承认这是一个“进程级唯一资源”，那就给它一个受控的全局访问点，同时尽量把它做得更像只读配置而不是可变状态。面试里更稳的表达是：我优先显式依赖（构造注入），只有当对象本质上就是全局资源时才考虑单例，并且尽量让单例保持不可变，避免“隐式共享可变状态”。

一个很实用的“识别信号”是：如果你发现到处都在传同一个对象、而且它的确代表进程级资源（配置、连接池、只读字典），你会自然想把它收敛成唯一入口；但如果你只是嫌参数传来传去麻烦，那往往说明你需要的是依赖注入，而不是单例。

适用场景	你确实需要“全局唯一”并且生命周期贯穿进程，例如配置读取器、进程级缓存（但要非常克制）
优点	访问点统一，避免重复初始化，省资源
代价	隐式全局状态，测试难，耦合高，容易引入并发与初始化顺序问题
常见误用	“为了方便”就上单例；把单例当成依赖注入容器；把可变状态塞进去导致难以回滚和复现问题

```python
from __future__ import annotations
from dataclasses import dataclass
from threading import Lock

@dataclass(frozen=True)
class AppConfig:
    db_dsn: str
    env: str

class ConfigSingleton:
    _instance: AppConfig | None = None
    _lock = Lock()

    @classmethod
    def get(cls) -> AppConfig:
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = AppConfig(db_dsn="sqlite:///:memory:", env="dev")
        return cls._instance
```

工厂方法模式（Factory Method）

区分一句话：**工厂方法**是“一个产品等级”的创建延迟到子类；**抽象工厂**是“成套产品族”一起替换（换主题/换供应商时整套切换）。

生活里你去点咖啡，通常不会关心“咖啡机内部怎么磨豆、怎么控温”。你只会说“给我一杯拿铁”。当店里换了咖啡机或换了配方，你依然想保持同样的点单方式。这其实是在强调：调用方应该依赖“我要什么”，而不是依赖“你怎么做”。

传统代码里，创建对象很容易写成分支：if 环境是 A 就 new AClient，环境是 B 就 new BClient。开始只有两三种实现时还好，后来实现越来越多，分支就散落在很多地方：某个接口里选一次，某个任务里又选一次。你会发现自己在维护“选择逻辑”，而不是在维护业务逻辑。

工厂方法的思路是：把“选择哪个具体实现”的知识集中起来，调用方只依赖抽象（接口/协议），创建这件事由工厂负责。这样当你新增实现或切换实现时，改动集中在工厂，调用方不被波及。

识别信号也很明显：当你看到调用方开始出现“根据条件选择实现”的分支，且这些分支在多个地方重复出现时，工厂方法几乎就是把这些分支收口到一个地方。

适用场景	创建哪个具体对象要由运行时条件决定，但创建逻辑又不想散在调用方里
优点	把“选择哪个实现”集中起来，调用方只依赖抽象
代价	多了一个间接层；如果类型过多需要管理注册表或映射

适用场景	创建哪个具体对象要由运行时条件决定，但创建逻辑又不想散在调用方里
常见误用	仅仅为了避免 `new` 就加工厂；把工厂写成巨大的 if-else

```python
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
from typing import Protocol

class Notifier(Protocol):
    def send(self, to: str, msg: str) -> None: ...

class EmailNotifier:
    def send(self, to: str, msg: str) -> None:
        print(f"[email] to={to} msg={msg}")

class SmsNotifier:
    def send(self, to: str, msg: str) -> None:
        print(f"[sms] to={to} msg={msg}")

class NotifierFactory:
    def create(self, channel: str) -> Notifier:
        if channel == "email":
            return EmailNotifier()
        if channel == "sms":
            return SmsNotifier()
        raise ValueError("unknown channel")
```

抽象工厂模式（Abstract Factory）

你装修房子时，往往不会“东拼西凑”买家具：沙发是北欧风，桌子是工业风，灯又是复古风，最后效果很难统一。更常见的做法是：选定一个“风格套系”，然后在这一套里买齐沙发、桌子、灯，它们天然是匹配的。

传统代码里，如果你要切换一整套实现（比如从 MySQL 切到 PostgreSQL，或者从 AWS 切到阿里云），很容易变成到处判断：这里选一个 Client，那边选一个 Repository，再那边选一个 Storage。你会遇到的真实问题是：你可能忘了某个点没切，导致一套实现被混搭，问题还很隐蔽。

抽象工厂的思路是：我不只创建“一个对象”，而是创建“一族彼此匹配的对象”。调用方只拿到抽象工厂，然后通过它拿到这一套里的各个产品，这样你切换时是一键切换整套，混搭风险大幅降低。

识别信号是“成套替换”：你不是只换一个组件，而是要换一整套彼此匹配的组件。如果你担心实现被混搭（半套 A + 半套 B），抽象工厂往往能把风险压下去。

适用场景	你要创建“一整套彼此匹配的对象族”，并且这套对象族有多种实现（例如不同数据库/不同 UI 主题）
优点	保证对象族一致性；切换“整套实现”时影响面更小
代价	抽象层变多；新增产品类型会比较痛（要改所有工厂）
常见误用	实际只有一个对象需要替换却上了抽象工厂，过度设计

建造者模式（Builder）

你去餐厅点“套餐”时，通常会被引导一步步选：主食选什么、饮料选什么、要不要加配菜。它和“你把所有选择一次性报完”相比，更不容易出错，也更不容易漏选。尤其当某些组合不允许时（比如某种主食不能配某种饮料），一步步引导更自然。

传统代码里，如果一个对象构造参数很多，最常见的结果是：构造函数越来越长，参数列表又长又容易传错，调用方看不出每个参数是什么含义。你可能会改成“传一个 dict”，结果可读性更差，错误也更晚暴露（运行时才发现缺字段）。

建造者模式的思路是：把“复杂创建过程”拆成一段可读的构建步骤，并且把最终的一致性校验集中在 `build()` 里。这样调用方写出来像一句话在描述对象怎么被组装出来，读起来清楚，错得也更早。

识别信号是“构造函数开始变得危险”：参数多到你需要不停翻文档确认顺序，或者对象的合法性取决于参数组合时，Builder 往往比继续加参数更稳。

适用场景	**一个对象构造参数很多、步骤多，且有“可选项 + 组合约束”，直接构造会让调用方很难用对**
优点	构建过程清晰；可以逐步构建并在最后统一校验不变量
代价	多引入一个 Builder 类型；对简单对象是额外负担
常见误用	把 Builder 当成“为了链式调用更酷”的语法糖；Builder 内不做校验导致最后仍可能构造出非法对象

```python
from dataclasses import dataclass
from decimal import Decimal

@dataclass(frozen=True)
class Invoice:
    customer_id: str
    amount: Decimal
    title: str
    tax_id: str | None = None

class InvoiceBuilder:
    def __init__(self) -> None:
        self._customer_id: str | None = None
        self._amount: Decimal | None = None
        self._title: str | None = None
        self._tax_id: str | None = None

    def customer(self, customer_id: str) -> "InvoiceBuilder":
        self._customer_id = customer_id
        return self

    def amount(self, amount: Decimal) -> "InvoiceBuilder":
        self._amount = amount
        return self

    def title(self, title: str) -> "InvoiceBuilder":
        self._title = title
```

```python
        return self

    def tax_id(self, tax_id: str) -> "InvoiceBuilder":
        self._tax_id = tax_id
        return self

    def build(self) -> Invoice:
        if not self._customer_id or self._amount is None or not self._title:
            raise ValueError("缺少必填字段")
        if self._amount < 0:
            raise ValueError("金额不能为负")
        return Invoice(
            customer_id=self._customer_id,
            amount=self._amount,
            title=self._title,
            tax_id=self._tax_id,
        )
```

原型模式（Prototype）

你做 PPT 时，经常会先复制一页“模板页”，再改标题和图片。你很少每次都从空白页重新拉框、调字号，因为那太浪费时间。复制的价值在于：模板里已经有大量默认设置，你只需要改动少量差异。

传统代码里，创建复杂对象如果每次都从头组装，会让你写出大量重复初始化逻辑。后来你可能开始“抽一个初始化函数”，但对象越复杂，这个初始化函数越像一个巨大的脚本，动一下就容易影响所有创建路径。

原型模式的思路是：我准备一个已配置好的“原型对象”，需要新对象时先复制，再做少量定制。注意它的风险也很真实：复制到底是深拷贝还是浅拷贝？如果原型里含有可变子对象（list、dict），浅拷贝会让新对象之间互相影响，这在面试里是非常好的“代价点”表达。

识别信号是“重复初始化”与“默认模板”：你发现创建过程里有一大段固定配置，每次只改少量差异，并且创建成本不低，那复制一个原型再微调往往更划算。

适用场景	创建成本高、并且你经常需要“从一个默认模板复制再微调”
优点	复制比重建更快；能保留复杂对象的默认配置
代价	深拷贝/浅拷贝容易踩坑；对象图复杂时复制语义难定义
常见误用	随手浅拷贝导致共享可变子对象，引发隐性联动 bug

结构型

适配器模式（Adapter）

你买了一个国外插头的电器，但家里墙上是国标插座。电器本身没问题，插座也没问题，问题是它们“接口形状不匹配”。你不可能为了这一个电器把全家插座重装，也不应该拿锤子硬敲插头。现实里你会买一个转换头：它不改变电器的内部，只是把接口翻译一下。

传统代码里接第三方库时经常发生这种事：第三方 SDK 的方法名、参数单位、返回结构与你系统的期望不一致。你最直觉的做法是在每个调用点做转换，结果转换逻辑被复制了 N 份，某天单位换算改了，你要全局搜、全局改，还容易漏。

适配器模式的思路是：我在边界处放一层“翻译官”，把外部接口转成内部期望的接口。调用方永远只认识内部接口，外部接口怎么变都尽量局部消化。面试里说得更直白一点：适配器不是为了复杂，是为了把“到处的转换”收敛到“一个地方”。

识别信号是“到处在做同一种转换”：单位换算、字段改名、返回结构拼装反复出现。你一旦发现自己写了第二次同样的转换，就该考虑把它抽成适配器。

适用场景	你要接入一个现成接口，但它的形状不符合你系统的期望，需要“翻译”一下
优点	不改旧代码就能接入新接口；调用方保持稳定
代价	多一层转发；适配层可能逐渐膨胀需要维护
常见误用	适配器里塞业务规则，导致边界被打穿

```python
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
from decimal import Decimal
from typing import Protocol

class LegacyPaymentSDK:
    def pay(self, cents: int) -> None:
        print(f"legacy pay {cents} cents")

class PaymentGateway(Protocol):
    def charge(self, amount_yuan: Decimal) -> None: ...

class LegacyPaymentAdapter:
    def __init__(self, sdk: LegacyPaymentSDK) -> None:
        self._sdk = sdk

    def charge(self, amount_yuan: Decimal) -> None:
        cents = int(amount_yuan * 100)
        self._sdk.pay(cents)
```

在工程里金额换算最好显式处理舍入（例如统一 `quantize` 到分），这里用 `int()` 只是为了让例子更短。

桥接模式（Bridge）

想象你在做一个“消息系统”：消息有多种类型（告警、营销、验证码），发送渠道也有多种（短信、邮件、App 推送）。如果你用继承表达，会很容易变成类爆炸：`AlertSmsSender`、`AlertEmailSender`、`MarketingSmsSender`……两个维度一乘，类数量就失控。

传统思路通常是“先凑合”：写一个大类，里面用 if-else 判断消息类型，再用 if-else 判断渠道。短期看简单，长期会出现两个问题：第一，大类越来越大，任何改动都可能影响别的分支；第二，扩展一个新渠道或新消息类型都要修改原来的大逻辑，违反开闭原则。

桥接模式的思路是把“抽象”和“实现”拆开：抽象这边是消息类型（它决定要说什么），实现这边是发送渠道（它决定怎么发）。两边通过组合连接，你新增消息类型不会影响渠道实现，新增渠道也不会影响消息类型。

识别信号是“两个变化轴互相乘起来”：你一边在加新类型，一边在加新实现，类/分支数量呈乘法增长。桥接用来把这两个轴拆开，各自扩展不互相拖累。

适用场景	**你有两个维度都在变化（例如“消息类型”与“发送渠道”），不希望用继承把它们乘起来**
优点	两个维度可以独立扩展；避免类爆炸
代价	抽象层变多；初看不如继承直观
常见误用	只有一个维度变化时也硬用桥接，增加理解成本

组合模式（Composite）

你看文件系统时，会把“文件夹”当成一个整体去复制、移动、算大小；而文件夹里又包含文件和子文件夹。你不会在操作时区分“这是文件夹所以要换一套 API”，你更希望“对文件和文件夹用同一种操作方式”。

传统代码如果用两套接口分别处理“叶子”和“容器”，调用方就会写大量分支：如果是文件就这么做，如果是文件夹就那么做。随着层级变深，处理逻辑变得递归又充满判断，读起来像一团麻。

组合模式的思路是：给“文件”和“文件夹”一个共同接口，让调用方只依赖这个接口；容器内部再递归调用子节点完成工作。这样对外就形成了“单个对象”和“对象集合”一致的使用体验，树形结构也被自然表达出来。

识别信号是“你希望对单个与一组用同一个操作”：调用方不停写 `if is_leaf else`，而且结构天然是树形/层级的，组合模式就很对味。

适用场景	**你要把对象组织成树形结构，并希望对“单个对象”和“对象集合”用同一套操作**
优点	统一对待叶子与容器；递归结构表达自然
代价	让类型边界变弱；某些操作对叶子/容器并不完全等价
常见误用	强行把不适合树结构的数据做成 Composite，导致 API 别扭

装饰器模式（Decorator）

你买手机壳时可以再贴膜，再加挂绳，再套一层防摔外壳。每个配件都不是“手机的一种子类”，但它们确实是在不改变手机本体的前提下，给手机增加能力，而且这些能力还可以叠加组合。

传统代码里，如果你用继承来做“能力叠加”，很快就会遇到组合爆炸：带日志的、带缓存的、带重试的、带限流的……继承树会像拼图一样越来越难维护。另一种直觉写法是在每次调用前后手写日志/缓存/重试，结果同样会带来重复与不一致。

装饰器模式的思路是：把“增强功能”做成一层层包装，核心对象只负责核心职责，增强能力按需叠加。它的代价也要承认：包装层太多会影响调试体验，所以要控制组合链长度，并让每一层职责非常单一。

识别信号是“横切能力要叠加”：日志、缓存、鉴权、重试这些能力既想复用又想组合。如果你开始考虑用继承来做各种组合，装饰器通常更合适。

适用场景	**你想在不改原类的情况下，给对象“叠加功能”，并且这些功能可以组合**
优点	组合灵活；比继承更可控；遵循开闭原则
代价	对象层级变深；调试时需要跟踪包装链
常见误用	用装饰器堆出一条很难读的调用链；装饰器里做与职责无关的事

```
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
```

```python
from typing import Protocol, Dict

class KVStore(Protocol):
    def get(self, key: str) -> str | None: ...
    def set(self, key: str, value: str) -> None: ...

class InMemoryStore:
    def __init__(self) -> None:
        self._d: Dict[str, str] = {}
    def get(self, key: str) -> str | None:
        return self._d.get(key)
    def set(self, key: str, value: str) -> None:
        self._d[key] = value

class LoggingStore:
    def __init__(self, inner: KVStore) -> None:
        self._inner = inner
    def get(self, key: str) -> str | None:
        print(f"[log] get {key}")
        return self._inner.get(key)
    def set(self, key: str, value: str) -> None:
        print(f"[log] set {key}")
        self._inner.set(key, value)
```

外观模式（Facade）

你去医院看病时，其实背后有挂号、分诊、检查、缴费、取药等很多环节。但你不希望每一步都自己拼装流程；你更希望有一个“导诊台”告诉你该去哪、下一步做什么。导诊台不一定更聪明，但它让你不用面对复杂系统的细节。

传统代码里，复杂子系统往往暴露一堆组件：订单要调用库存、支付、物流、通知。调用方（比如接口层）为了完成一次业务操作，会同时知道很多内部组件的存在与顺序。结果是：调用方变得很臃肿，流程规则散落，而且任何底层组件调整都会波及大量调用点。

外观模式的思路是：在子系统外面提供一个更粗粒度的入口，把复杂协作封装进去。调用方只面对少数几个“我想做什么”的方法。面试里你可以强调：外观不等于上帝对象，外观最好只做编排与协调，把业务规则仍留在领域层。

识别信号是“调用方被子系统细节淹没”：为了完成一件事，调用方要知道很多组件和调用顺序。外观把这件事变成一个清晰入口，让边界更好守。

适用场景	子系统很复杂，但调用方只需要一条“简单入口”完成一个任务
优点	降低使用成本；隐藏复杂协作；让边界更清晰
代价	外观可能变成“万能门面”需要克制；底层变化仍可能渗透出来
常见误用	Facade 里塞进大量业务逻辑，最后成上帝对象

享元模式（Flyweight）

你在地图应用里拖动地图时，屏幕上可能同时出现成千上万个小图标与瓦片。如果每个图标都创建一份完整的颜色、字体、图片数据，内存会迅速膨胀。现实里更常见的做法是：把“可共享的资源”放在一个缓存里，重复使用同一份。

传统代码面对“大量相似对象”时，常见后果是内存和 GC 压力变大，性能抖动明显。你可能会加缓存，但如果没弄清楚“哪些状态可以共享，哪些必须独立”，缓存会变得危险：共享了不该共享的可变数据，反而引入隐性 bug。

享元模式的思路是区分两类状态：内蕴状态（可共享、通常不可变）放到享元对象里缓存；外蕴状态（每次都不同）由调用方在使用时提供。面试里讲清这个区分，就把享元的核心讲清了。

识别信号是“对象数量巨大且高度相似”：你在 profiler 里看到大量重复数据结构占内存，而其中很多字段在不同实例之间完全相同。这时把可共享部分缓存起来会很值。

适用场景	大量相似对象导致内存压力，且其中大部分状态可以共享（例如字符、棋子纹理、地图瓦片）
优点	大幅节省内存；提升缓存命中
代价	需要区分内蕴/外蕴状态；引入缓存管理复杂度
常见误用	仅为了“看起来高级”引入缓存，结果并没有真正的规模问题

代理模式（Proxy）

你在公司内网访问一个敏感系统时，经常不会直接连到它，而是先经过网关：网关会做鉴权、限流、审计，甚至会在后面做缓存与熔断。你最终访问的是“系统”，但你先经过了一个“代表它的入口”。

传统代码里，如果你把鉴权、限流、懒加载、远程调用这些控制逻辑散在每个调用点，重复是小事，更大的问题是你很难保证策略一致。某个地方忘了鉴权、某个地方没有限流，问题就变成安全事故或线上故障。

代理模式的思路是：我提供一个和真实对象同形的替身，在这个替身里做访问控制，然后再把调用转给真实对象。它和装饰器很像，但关注点不同：代理更强调“访问前后的控制/间接访问”，装饰器更强调“能力增强与组合”。

识别信号是“访问前后需要控制”：权限、限流、懒加载、远程调用、审计。如果你发现这些控制逻辑散落在多个调用点，代理可以把它们集中起来。

适用场景	你需要在访问对象前后加控制：懒加载、权限校验、远程调用、限流等
优点	在不改被代理对象的情况下增加控制能力
代价	多一层间接；容易和装饰器混淆需要讲清差别
常见误用	代理层太“聪明”，把业务逻辑吸进去

```python
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
class ExpensiveReport:
    def render(self) -> str:
        print("generate report ...")
        return "REPORT"

class LazyReportProxy:
    def __init__(self) -> None:
        self._real: ExpensiveReport | None = None

    def render(self) -> str:
        if self._real is None:
            self._real = ExpensiveReport()
        return self._real.render()
```

行为型

责任链模式（Chain of Responsibility）

你在公司报销时，单据往往会按金额大小走不同审批链：小额可能组长就能批，大额要经理、财务、甚至总监。你提交的其实是一份“请求”，它会沿着一条链路往前走，直到被处理或者被拒绝。你不希望自己写死“先找谁再找谁”，因为审批规则经常变。

传统代码最容易写成一段又长又脆的 if-else：金额 < 1000 就 A 批；否则 B 批；如果是差旅还要走 C；如果是紧急还要插队。规则一多，分支就缠绕在一起，而且很难插入一个新的步骤（比如加一个风控检查）。

责任链模式的思路是把每个检查/处理步骤拆成独立节点，然后把节点按顺序串起来。请求进入链路后，节点要么处理它，要么把它交给下一个节点。这样你改变规则通常只需要调整链的组合或顺序，不必把所有逻辑揉在一个大函数里。

识别信号是“规则像步骤一样可插拔”：校验、过滤、审批、风控这些通常是一串步骤，而且步骤会增删、会调顺序。责任链就是让步骤变成可组合的积木。

适用场景	一个请求要经过一系列处理者，具体走哪些处理者取决于条件（例如审批流、校验链、过滤器）
优点	处理者可插拔；顺序可调整；调用方只管“交给链”
代价	调试时要跟踪链路；链配置不当可能吞掉请求
常见误用	链里每个节点都做太多，导致“链”变成“巨大流程”

```python
from typing import Callable, Optional

Handler = Callable[[dict], Optional[str]]

def auth(req: dict) -> Optional[str]:
    return None if req.get("user") else "unauthorized"

def validate(req: dict) -> Optional[str]:
    return None if req.get("amount", 0) > 0 else "bad request"

def handle(req: dict) -> Optional[str]:
    for h in (auth, validate):
        err = h(req)
        if err:
            return err
    return None
```

命令模式（Command）

你点外卖时，下单这件事会被变成一张“订单”：它可以排队（骑手接单前先排队）、可以重试（支付失败可以再付）、可以撤销（未出餐可取消）、可以审计（平台能回放你做过什么）。关键点是：下单不再只是一个“函数调用”，它变成了一个可存、可传、可重放的东西。

传统代码里，业务操作常常只是一个方法调用：`place_order()`。当你需要把它放进队列、做重试、做审计、做延迟执行时，你会发现“函数调用”不够装载这些需求，你开始往函数里塞各种参数与分支，结果越来越难维护。

命令模式的思路是把“一个操作”封装成命令对象，命令对象携带执行所需的数据，并提供统一的 `execute()` 。这样你就能把命令排队、持久化、重试、撤销或记录日志。面试里你可以用这句话收尾：命令模式把“做什么”变成一等公民。

识别信号是“操作需要被当作数据处理”：要排队、要重试、要延迟、要撤销、要审计。此时把操作封装成命令对象，往往比把这些能力硬塞进函数更干净。

适用场景	**你要把“一个操作”封装成对象/数据：可排队、可撤销、可重放（例如任务队列、按钮操作、事务脚本）**
优点	解耦调用方与执行方；支持日志、撤销、重试、审计
代价	命令对象增多；需要管理上下文与参数
常见误用	把所有函数都包一层命令，结果复杂度大增却没有收益

```python
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
from dataclasses import dataclass
from typing import Protocol, List

class Command(Protocol):
    def execute(self) -> None: ...

@dataclass
class SendEmail:
    to: str
    msg: str
    def execute(self) -> None:
        print(f"send email to={self.to}: {self.msg}")

class CommandBus:
    def __init__(self) -> None:
        self._queue: List[Command] = []
    def enqueue(self, cmd: Command) -> None:
        self._queue.append(cmd)
    def run(self) -> None:
        while self._queue:
            self._queue.pop(0).execute()
```

解释器模式（Interpreter）

你在 Excel 里写公式时，其实就是在写一门小语言： `=SUM(A1:A10)` 。你不需要写程序去循环相加，你只写表达式，系统负责解析和执行。很多业务系统也会有类似需求：运营想写“规则表达式”，开发不想每次改规则都发版。

传统做法通常是把规则写死在 if-else 里。开始还好，后来需求变成“规则可配置、可组合、可解释”，if-else 立刻崩溃。你可能会改成“用字符串描述规则然后手写解析”，但如果没有明确的语法结构，解析会越来越像一坨混乱的正则和切字符串。

解释器模式的思路是：把语法结构显式化（表达式树），每个语法节点知道如何解释自己。它的现实代价也要说清楚：真正复杂的语言不适合手写解释器，生产中更常用解析器生成器或规则引擎。面试里“为什么少用”就回答：因为复杂语法的成本高，只有在语法很小、变化可控时解释器才划算。

识别信号是“规则要被配置而不是写死”：当业务开始要求可配置表达式、可组合条件、可解释原因时，你就需要把规则从 if-else 里抬出来，解释器是其中一种做法。

适用场景	**你有一套小语言/表达式，需要解析并执行（例如规则表达式、查询过滤、DSL）**
优点	把语法结构显式化；扩展表达式类型相对直观
代价	类数量容易膨胀；复杂语法更适合用解析器生成器或现成库
常见误用	明明只是简单 if-else，却硬上解释器；或用解释器去实现完整语言导致维护灾难

迭代器模式（Iterator）

你去图书馆找书时，并不需要知道图书馆内部的存放结构是“按楼层、按书架、按编号”。你只需要一个“遍历方式”：下一本、下一本、下一本。遍历这件事被抽象成了一个统一接口，内部结构被隐藏了。

传统代码如果把集合内部结构暴露给调用方，调用方就会依赖太多细节：这是 list 还是 dict？要不要按某个顺序？当内部结构变化时，调用方就被迫一起改。更糟糕的是，有些遍历应该是惰性的（比如大文件流式读取），如果你一口气把所有东西展开，会带来内存与性能问题。

迭代器模式的思路是：集合提供一个统一的遍历器，调用方只依赖“能否遍历”，不依赖内部表示。在 Python 里这几乎就是语言内置的迭代协议。面试里你只要说清“统一遍历 + 隐藏内部结构 + 支持惰性”就够了。

识别信号是“遍历方式想保持稳定，但内部结构可能变”：调用方只想 `for x in ...`，不想知道底层是分页、流式读取还是树形展开。迭代器把这些细节藏起来。

适用场景	**你希望以统一方式遍历集合，但不暴露集合内部结构**
优点	遍历逻辑与集合结构解耦；支持惰性遍历
代价	对某些简单集合是多余；自定义迭代器要处理状态
常见误用	为了“有模式”而自己写迭代器，忽略语言自带迭代协议

中介者模式（Mediator）

你组织一次多人聚餐，如果每个人都要分别和其他人沟通时间、地点、忌口，沟通成本会指数级上升。更好的方式是建一个群，或者找一个组织者：大家只和组织者沟通，组织者再协调整体。这并不是让组织者更强大，而是让人与人之间的直接依赖减少。

传统代码里，当多个对象互相调用、互相通知，会形成网状耦合。最开始可能只是 A 调 B、B 调 C；后来变成 A 既调 B 又调 C，C 又要调 A，逻辑在对象之间来回跳。你会发现修改任意一个对象的行为都可能影响一大片。

中介者模式的思路是把交互规则集中到一个中介者里：对象之间不直接对话，而是通过中介者转发与协调。它的代价也要讲清：中介者可能膨胀成上帝对象，所以要限制职责范围，让中介者只负责“协调”，不要吞掉业务规则。

识别信号是“对象之间形成网状耦合”：A 知道 B，B 又知道 C，C 又回头知道 A。你越改越像牵一张蜘蛛网，中介者用来把沟通收口。

适用场景	**多个对象之间交互关系复杂、网状耦合严重，希望把交互集中到一个中介者**
优点	降低对象间直接依赖；交互规则集中
代价	中介者可能膨胀成上帝对象；需要治理复杂度

适用场景	多个对象之间交互关系复杂、网状耦合严重，希望把交互集中到一个中介者
常见误用	用中介者把所有业务都堆进去，最终更难改

备忘录模式（Memento）

你在写文档时按下“撤销”，系统就能回到上一版。你并不希望每次修改都手动记录“我改了哪一行”，你更希望系统在关键时刻帮你保存一个快照。这就是“保存状态以便恢复”的需求。

传统代码里，如果你用一堆临时变量记录“之前是什么”，很快会失控：字段越多，要记的越多；对象关系越复杂，要记的越复杂。你可能会把整个对象序列化保存，但又会遇到体积大、频率高导致性能问题。

备忘录模式的思路是：对象对外不暴露内部细节，但可以生成一个“备忘录”快照；需要恢复时再把备忘录交回对象。面试里你要强调真正的工程点：快照的边界与粒度，以及保存频率，不然撤销功能可能反而拖垮系统。

识别信号是“需要撤销/回滚，但不想暴露内部结构”：对象越复杂，手动记录差异越不现实。备忘录让状态快照成为对象自身的能力。

适用场景	需要保存并恢复对象历史状态，但又不想暴露内部结构（例如撤销/回滚）
优点	支持撤销；状态封装良好
代价	状态快照可能很大；需要定义快照边界
常见误用	频繁保存全量快照导致性能/内存问题；快照里混入外部资源句柄

观察者模式（Observer）

你关注一个公众号，它发新文章时，你会收到推送。公众号并不需要知道你是谁、你在哪个手机上，它只需要“发布事件”。订阅者也不需要知道公众号内部怎么生成内容，它只需要在被通知时执行自己的动作。

传统代码里，如果你在业务流程里硬编码“订单支付后要做 A、B、C”，它很快会变成一个越来越长的清单。你想加一个新功能（比如埋点、通知、发票），就必须改支付流程代码，这会让核心流程越来越不稳定。

观察者模式的思路是：发布者只负责发布事件，订阅者自行订阅并处理。这样你新增订阅者时不需要改发布者。代价也要说清：事件流很难追踪，尤其当订阅者很多、链路很长时，所以要有规范（幂等、失败处理、可观测性）。

识别信号是“事后动作越来越多”：核心流程稳定，但每次都要新增一个‘支付后做什么’。观察者把这些事后动作变成可订阅的扩展点。

适用场景	一个对象状态变化，需要通知多个订阅者（例如事件总线、UI 更新、领域事件）
优点	发布者与订阅者解耦；扩展订阅方容易
代价	事件流难追踪；顺序/重复通知/异常处理要规范
常见误用	观察者里做重业务导致事件链难控；滥发事件导致系统“抖动”

```python
from typing import Callable, List

class EventBus:
    def __init__(self) -> None:
        self._subs: List[Callable[[str], None]] = []

    def subscribe(self, fn: Callable[[str], None]) -> None:
        self._subs.append(fn)

    def publish(self, event: str) -> None:
        for fn in list(self._subs):
            fn(event)
```

状态模式（State）

你处理“订单”时会发现：不同状态下允许做的事完全不同。未支付可以取消，已支付可以退款，已发货只能退货。最容易写出来的代码，就是在每个操作里写一堆 if-else 判断当前状态。

传统写法的问题不在“写不出来”，而在于规则变多后，你会在多个方法里重复状态判断：`cancel()` 里判断一次，`ship()` 里判断一次，`change_address()` 里再判断一次。某天新增一个状态（比如部分发货），你得改很多地方，还容易漏。

状态模式的思路是把“状态对应的行为”封装到状态对象里，让对象在不同状态下委托给不同状态实现。这样你把巨大的 if-else 拆成了清晰的状态类。面试里你可以补一句取舍：状态少时放在实体方法里就够；分支爆炸时再上状态对象，避免过早复杂化。

识别信号是“同一个对象在不同状态下行为完全不同”：if-else 开始蔓延到各处，并且状态转换规则也越来越多。状态模式把状态机显式化，减少分支纠缠。

适用场景	对象行为随状态变化而变化，并且状态转换有明确规则（例如订单状态机、审批状态机）
优点	用状态对象替代巨大 if-else；状态与行为绑定更清晰
代价	状态类增多；需要管理转换关系
常见误用	状态过细导致类爆炸；或状态转换逻辑散落在各状态里没人能看全局

策略模式（Strategy）

你选择出行方式时，会根据场景切换：距离近就走路，赶时间就打车，预算紧就坐地铁。核心目标是“到达目的地”，但实现方式有多种，而且可以在运行时选择。

传统代码把这些选择写成 if-else：如果距离 < 1km 走路，否则如果预算 < 20 坐地铁，否则打车。规则越来越多时，这个 if-else 会变得难以维护，而且测试也很痛苦：你要覆盖每个分支组合。

策略模式的思路是把“算法/规则”抽成策略接口，每种实现写成一个策略类/函数。主流程只依赖策略抽象，选择策略的逻辑集中管理。你在面试里可以顺势说：策略让扩展更轻，但也带来一个管理成本——策略怎么选、在哪选、如何配置。

识别信号是“算法/规则可替换”：你在做同一件事，但实现方式有多种，且经常新增/替换。把算法抽成策略，主流程就能保持稳定。

适用场景	一段算法/规则有多种实现，且可以在运行时切换（例如定价策略、路由策略、排序策略）
优点	替换简单；开闭；便于测试每个策略
代价	需要管理策略选择；多一个抽象层
常见误用	策略接口过于“万能”，导致每个策略要接很多无关参数

模板方法模式（Template Method）

你做菜时，“炒一盘菜”的流程很固定：备料、下锅、调味、出锅。但不同菜的差别主要在某几步，比如调味配方不同、火候不同。你不会每次都重新发明“炒菜流程”，你只想复用骨架，并在少数步骤上定制。

传统代码里如果把流程复制粘贴到多个类/函数里，后面你想改一个流程步骤（比如加一个清洗步骤）就要改很多处。你也可能抽一个大函数，然后用回调参数来控制差异，但如果差异点多，回调会变得难读。

模板方法模式的思路是固定流程骨架，把变化点留给子类覆盖（或在 Python 里用钩子方法/回调实现）。面试里你可以加一层现实取舍：它常用继承实现，所以要注意继承的边界；在 Python 中也可以用组合或函数参数替代继承，表达同样的思想。

识别信号是“流程骨架稳定、步骤可变”：你反复写相同流程，只是某几步不同。模板方法把骨架固定住，把变化点留成钩子。

适用场景	一套流程固定，但其中几步需要延迟到子类/实现去填（例如导入流程、任务流水线）
优点	固定骨架，变化点可覆盖；避免重复流程代码
代价	常用继承实现，容易受继承缺点影响；在 Python 里也可用回调替代
常见误用	把模板方法用成“超大框架”，子类只能在缝里写代码导致难维护

```python
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
from abc import ABC, abstractmethod

class ImportJob(ABC):
    def run(self) -> None:
        raw = self.read()
        items = self.parse(raw)
        self.save(items)

    @abstractmethod
    def read(self) -> str: ...

    @abstractmethod
    def parse(self, raw: str) -> list[str]: ...

    def save(self, items: list[str]) -> None:
        print(f"save {len(items)} items")

class CsvImport(ImportJob):
    def read(self) -> str:
        return "a,b,c"
    def parse(self, raw: str) -> list[str]:
```

```
        return raw.split(",")
```

访问者模式（Visitor）

你面对一棵“语法树”时，经常会对它做很多不同操作：计算表达式值、生成代码、做静态检查、输出格式化文本。语法树结构通常比较稳定，但操作会不断新增。你不希望每新增一个操作就去改语法树的每个节点类。

传统做法是把操作写进节点类里：节点既负责数据结构，又负责各种算法。时间久了节点类会越来越肥，而且新增一个新操作就要改很多节点类，风险很大。另一种做法是在外面写很多 `isinstance` 分支判断节点类型，这同样会变得难维护。

访问者模式的思路是：数据结构负责“接受访问者”，具体算法由访问者对象提供。你新增一种操作就新增一个访问者，不用改节点结构。它的代价也很明确：如果你新增一种节点类型，就要修改所有访问者，所以它适合“结构稳定、操作多变”的场景。

识别信号是“结构稳定、操作多变”：你有一棵结构（比如 AST），但总在加新操作（检查、导出、计算）。访问者让你新增操作时不必改结构本身。

适用场景	你要对一组稳定的数据结构做很多种操作，并且操作会不断新增（例如语法树分析、报表导出）
优点	新增操作不改数据结构；把算法集中到访问者
代价	新增数据结构类型很痛（要改所有访问者）；双分派理解成本高
常见误用	数据结构并不稳定却用访问者，导致每次加节点都要全局修改

为了让你在面试里快速对齐“这个模式和那个模式有什么区别”，这里给一个简短的对照表。它不是为了背，而是为了你讲的时候能把边界说清。

容易混淆	你可以这样区分
代理 vs 装饰器	代理强调“访问控制/间接访问”，装饰器强调“能力叠加/功能增强”
策略 vs 状态	策略是“可替换算法”，状态是“状态驱动行为并且有转换规则”
外观 vs 适配器	外观是“简化入口给我用”，适配器是“把你的接口翻译成我想要的形状”
工厂方法 vs 抽象工厂	工厂方法通常解决“创建一个产品的不同实现”，抽象工厂解决“创建一族彼此匹配的产品”

面试高频加分块

这一章会把“工程能力”用少量主题覆盖到：可读性与重构、错误处理、并发一致性、测试策略。目标是让你在追问里有稳定的表达结构，不需要临场发挥。

代码可读性与重构

面试里聊“可读性与重构”，很多人会把它讲成一堆原则：短函数、少 if、别重复。原则没错，但面试官真正想听的是：你怎么在真实代码里判断“这里开始危险了”，以及你怎么在不把系统搞崩的前提下，把它慢慢修好。

先从一个非常具体的现象开始：你打开一个函数，往下滚三屏都没到底；或者你看到三层嵌套 if，脑子里已经开始忘记最外层条件是什么；再或者你发现同一段计算逻辑被复制了四份，只是变量名不一样。传统思路往往是“忍一忍，先交付”，因为改起来怕出事故。结果是这段代码会变成团队的“雷区”：谁都不敢碰，但需求还是会不断往上堆。

重构的思路不是“推倒重写”，而是“在可控范围内持续减少风险”。一个很好用的节奏是：先用测试或最小校验把行为钉住，再做小步改动，每一步都能回滚、都能验证。很多时候你不需要一口气把它改完，你只要让它从“不可读、不可改”变成“能读、能动”，剩下的可以在后续迭代继续变好。

下面用一个小例子展示“长函数 + 重复逻辑”的典型重构方式。注意重点不是代码技巧，而是思路：先把意图命名出来，再把重复收敛成单一入口，最后让主流程读起来像叙述。

```python
# 重构前：流程挤在一起，规则散落，重复也看不清
def price_order(user_level: str, items: list[tuple[str, int]], coupon: str | None) -> int:
    total = 0
    for sku, qty in items:
        if sku.startswith("A"):
            total += 100 * qty
        else:
            total += 80 * qty

    # VIP 折扣
    if user_level == "vip":
        total = int(total * 0.9)

    # 优惠券折扣（重复：又在写折扣逻辑）
    if coupon == "10OFF":
        total -= 10
    elif coupon == "20OFF":
        total -= 20

    if total < 0:
        total = 0
    return total


# 重构后：把“变化点”抽出来，主流程更像一句话
def base_total(items: list[tuple[str, int]]) -> int:
    def unit_price(sku: str) -> int:
        return 100 if sku.startswith("A") else 80

    return sum(unit_price(sku) * qty for sku, qty in items)

def apply_user_discount(total: int, user_level: str) -> int:
    return int(total * 0.9) if user_level == "vip" else total

def apply_coupon(total: int, coupon: str | None) -> int:
    if coupon is None:
```

```
        return total
    mapping = {"10OFF": 10, "20OFF": 20}
    return total - mapping.get(coupon, 0)

def clamp_non_negative(total: int) -> int:
    return max(total, 0)

def price_order_refactored(user_level: str, items: list[tuple[str, int]], coupon: str | None) -> int:
    total = base_total(items)
    total = apply_user_discount(total, user_level)
    total = apply_coupon(total, coupon)
    return clamp_non_negative(total)
```

在面试里你可以把这段话说得很“工程”：我不是为了优雅而重构，我是为了减少认知负担和改动风险；我会先钉住行为，再小步收敛重复、减少嵌套、让主流程像叙述。最后再补一句“童子军规则”：每次改动顺手把附近变好一点，长期效果非常明显。

错误处理与边界

错误处理是后端面试里很容易出区分度的地方，因为它不仅是“try/except 怎么写”，更是“你把失败当成什么”。一个很好用的起点是把失败分成两类：**预期失败**（比如用户余额不足、参数不合法、库存不够）和 **非预期失败**（比如网络超时、依赖服务 500、程序 bug）。这两类失败的处理方式应该完全不同。

传统思路常见两种极端：一种是所有失败都抛异常，调用方只能靠 try/except 分辨；另一种是所有失败都返回 None/False，让错误信息丢失，排障困难。更现实的坏味道是“重试风暴”：只要失败就重试，多个服务一起重试，最后把本来还能扛住的依赖打崩。

更稳的思路是：预期失败属于正常分支，应该被清晰表达（返回结果对象或业务异常，并带上可读的错误码/原因）；非预期失败需要被记录、告警，并且通常要配合重试、降级、熔断等策略。重试不是万能药，它要和**幂等性**绑定在一起：如果一次操作重试会产生重复扣款、重复发货，那重试就是灾难。工程上常见做法是引入幂等键（idempotency key），让同一请求无论重试多少次，最终效果都只发生一次。

下面给一个很小的“结果对象 + 幂等键”的写法。它不是框架，只是为了让你在面试里能把“失败是正常分支还是异常”讲清楚。

```
# 注：此代码块可能含占位符（...），用于说明结构；如需运行请补齐实现
from dataclasses import dataclass
from typing import Generic, TypeVar, Optional, Dict

T = TypeVar("T")

@dataclass(frozen=True)
class Result(Generic[T]):
    ok: bool
    value: Optional[T] = None
    error_code: Optional[str] = None
    message: Optional[str] = None

    @staticmethod
    def success(value: T) -> "Result[T]":
        return Result(ok=True, value=value)
```

```python

    @staticmethod
    def fail(code: str, msg: str) -> "Result[T]":
        return Result(ok=False, error_code=code, message=msg)


class IdempotencyStore:
    # 真实项目可用 Redis/DB，这里用内存示意
    def __init__(self) -> None:
        self._done: Dict[str, str] = {}

    def seen(self, key: str) -> bool:
        return key in self._done

    def mark_done(self, key: str, outcome: str) -> None:
        self._done[key] = outcome


def charge(idempotency_key: str, amount: int, store: IdempotencyStore) -> Result[str]:
    if amount ≤ 0:
        return Result.fail("INVALID_AMOUNT", "金额必须为正数")

    if store.seen(idempotency_key):
        # 重试命中：直接返回已完成结果
        return Result.success("already-charged")

    # 假装这里调用第三方支付，可能超时/失败（非预期失败）
    try:
        # third_party.pay( ... )
        store.mark_done(idempotency_key, "charged")
        return Result.success("charged")
    except TimeoutError:
        # 这里不吞掉异常：记录/告警后向上抛，让上层按策略重试
        raise
```

面试里你可以用一句话收尾：我区分预期失败与非预期失败，预期失败清晰表达并可恢复，非预期失败可观测并配合重试/降级；重试必须建立在幂等性上，否则会把偶发故障放大成数据事故。

在国内面试里，如果你能多补一句“可观测性”，往往会直接加分：非预期失败不仅要处理，还要能被看见。最小闭环是日志（带上下文与请求标识）、指标（成功率/超时率/重试次数）、告警（阈值触发）、必要时加链路追踪（定位哪一跳慢/错）。尤其对重试而言，监控“重试次数”和“退避后的总体延迟”非常关键，否则你以为在自愈，实际在悄悄放大故障。

如果继续往“线上稳定性策略”追问，你可以用一个很常见、也很好背的组合顺序回答：**先超时，再重试，配退避与抖动；必要时熔断保护下游，再用限流保护自己。**超时是为了避免线程/连接被无期限占住；重试要指数退避并加随机抖动，避免大家同一时间一起重试；熔断是在下游已经不健康时快速失败，留资源给可用请求；限流是防止自身被洪峰打穿。最后别忘了把它们和可观测性绑在一起：你要监控超时率、重试次数、熔断打开次数、限流丢弃量，否则策略是否有效你其实不知道。

并发与一致性

并发与一致性最容易讲虚，因为很多人会直接背“加锁、用队列、原子操作”。但面试官真正想听的是你有没有那个直觉：**风险源在共享可变状态**。只要多个执行单元（线程/协程/进程/多实例服务）会同时读写同一份可变数据，你就必须回答两个问题：谁能写？写的时候怎么同步？如果同步失败会发生什么？

传统思路常见两种：一种是“先不管，应该没那么巧同时发生”，这在压测和线上迟早会翻车；另一种是“一把大锁锁住一切”，它能保正确，但可能把吞吐锁死，并且容易导致死锁或锁竞争。更稳的思路是先保证正确性，再逐步优化：尽量减少共享、把可变状态关进更小的范围，能用不可变就别共享可变，能用消息队列串行化写就别多点写。

下面用一个最经典的例子解释“为什么会错”。同一份计数器，在两个线程里各加十万次，看起来应该是二十万，但在没有同步时，最终结果往往更小，因为 `x += 1` 并不是原子操作。

```python
import threading

counter = 0

def add(n: int) -> None:
    global counter
    for _ in range(n):
        counter += 1

t1 = threading.Thread(target=add, args=(100_000,))
t2 = threading.Thread(target=add, args=(100_000,))
t1.start(); t2.start()
t1.join(); t2.join()

print(counter)  # 常见情况下会小于 200000（是否复现取决于调度时机）
```

补充一句避免被抬杠：Python 的 GIL 并不等于“不会有竞态”，它只是不让多个线程同时执行字节码；对“读-改-写”这种复合操作，仍可能在不同调度点发生交错。所以这个例子用来强调的是并发推理的原则，而不是“每次必现”。

最直接的修复是锁。锁的价值不在于“我会写 lock”，而在于你知道锁在保护什么：保护的是那段读-改-写的临界区。

```python
import threading

counter = 0
lock = threading.Lock()

def add(n: int) -> None:
    global counter
    for _ in range(n):
        with lock:
            counter += 1
```

但工程里你通常不会满足于“处处加锁”。更常见的思路是用队列把写入串行化：所有写操作都交给一个消费者来做，其他线程只负责把任务放进队列。这样你用“结构”消灭了共享写，而不是用“到处的锁”去补救。

并发下的一致性也常常不是“强一致/弱一致”的学术讨论，而是业务直觉：在扣库存、扣余额、发货这种场景里，正确性优先于性能；你宁愿慢一点，也不要出现负库存或重复扣款。面试里讲清这一点，会比背一堆名词更有说服力。

如果面试官把话题往“性能与容量”上带，你可以用一个稳的顺序回答：先保正确，再做热点优化。常见优化手段其实都在结构层面：减少锁粒度、把共享写串行化、用缓存减少 I/O、批处理减少往返、异步化把非关键路径移出去。你不需要报一堆名词，只要能说明“我会先用数据找热点，再选最小风险的优化点”，就已经很工程。

测试策略

测试策略这一节的关键，是把它从“写单测”讲成“怎么让系统变得可测”。可测试性不是测试框架给的，是结构给的：边界清晰、依赖可替换、核心逻辑不被 I/O 绑架。你在第二章引入 DI，其实就是在为测试铺路——这也是面试官很爱追问的点。

传统思路里最常见的误区是“所有东西都写端到端测试”。它当然重要，但它慢、脆、排障难：失败了你不知道是逻辑错、环境错还是数据错。更稳的策略是测试金字塔：大多数用单测覆盖核心逻辑与规则；少量集成测试验证边界（数据库、消息、HTTP）；极少量端到端测试覆盖关键链路。不是因为单测高贵，而是因为成本结构不同。

Mock 也是一个很容易踩坑的地方。很多人把 Mock 当成“随便替换”，结果把内部实现细节也 mock 掉了，测试变得和实现绑定，重构一下就全红。更稳的做法是：只 mock 不稳定或外部依赖（时间、网络、数据库、第三方），不要 mock 你自己写的纯逻辑。换句话说，Mock 应该站在边界上，而不是站在核心里。

下面用一个最小例子展示“边界清晰 + DI”带来的单测体验：用例依赖仓储协议，测试时用内存仓储，不需要启动数据库。

下面的测试片段为了突出“边界清晰 + DI 带来可测性”，默认你已经有前文的 `Order` 、 `Address` 、 `Money` 、 `InMemoryOrderRepo` 、 `PricingService` 等定义（真实项目里它们来自你的业务模块）。

```python
from decimal import Decimal

def test_submit_order_happy_path():
    # 准备：用内存仓储
    repo = InMemoryOrderRepo()
    factory = OrderFactory()
    order = factory.create(user_id="u1", address=Address(city="SH", detail="road 1"))
    order.add_item("A001", 1)
    repo.save(order)

    # 准备：用可控的 catalog/discount
    class FixedCatalog:
        def price_of(self, sku: str) -> Money:
            return Money(Decimal("100.00"))

    class NoDiscount:
        def apply(self, total: Money) -> Money:
            return total

    pricing = PricingService(catalog=FixedCatalog(), discount=NoDiscount())
    usecase = SubmitOrderUseCase(repo=repo, pricing=pricing)

    # 执行
    usecase.execute(order.id)

    # 断言：领域规则被正确触发
    saved = repo.get(order.id)
```

```
    assert saved is not None
    assert saved.status.value == "submitted"
    assert saved.total.amount == Decimal("100.00")
```

面试里你可以用一句话收尾：我用结构换可测性——把规则放在领域对象里，把 I/O 放在边界外，把依赖通过 DI 注入；然后用测试金字塔控制成本，用 Mock 守住边界不侵入核心。